

**Lab Title: SYN Flood Attack Investigation
Using tshark**

Student Name: Samuel Chinedu Onu

Student ID: 2025/FWSD/11518

**Course Name: SBT-DF203: Basic
Networking Skills for Digital Forensics**

Instructor Name: Aminu Ibrahim Jalo

Date of Submission: 11th September 2026

International Cybersecurity and Digital Forensics Academy (ICDFA)

SBT-DF203: Basic Networking Skills for Digital Forensics

LAB 3: SYN Flood Attack Investigation Using tshark

Three-Week Delivery Block 1/3 | 5-11 September 2026

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 3
Lab Title	SYN Flood Attack Investigation Using tshark
Delivery Block	1/3 of 3
Scheduled Dates	5-11 September 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	mySYNFloodCapture.pcap or a bounded loopback capture generated in the lab

1. Source Materials and Slide Alignment

- HTTP, tshark and SYN Flood: TCP three-way handshake, half-open connections, SYN flood concept, Scapy packet generation and tshark capture.
- The uploaded slide uses Apache on port 80, loopback capture and a four-packet controlled simulation.
- The assignment requires two authorized VMs or an equivalent isolated environment and evidence that distinguishes attack-like SYN activity from a normal handshake.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

An Apache service experienced repeated TCP SYN packets that were not followed by the final ACK of the three-way handshake. You will preserve the packet capture, establish a normal-traffic baseline, identify suspicious SYN patterns, calculate indicators, and conduct a strictly bounded loopback simulation to validate your interpretation.

3. Learning Outcomes

- Differentiate a complete TCP handshake from an incomplete or half-open attempt.
- Use tshark display filters and field extraction to identify SYN, SYN-ACK, ACK and reset packets.
- Calculate SYN counts, unique source ports, response behaviour and incomplete-handshake indicators.
- Create a four-packet loopback-only simulation with Scapy in an authorized environment.
- Develop a defensible timeline and incident finding from PCAP evidence.
- Recommend controls for detecting and mitigating SYN flood activity.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The simulation is limited to four SYN packets against 127.0.0.1 or an instructor-assigned private lab IP. Do not increase packet count, automate continuous sending, spoof third-party addresses or target public systems.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux VM	Run tshark, Apache and Scapy.
Target service	Local Apache on 127.0.0.1:80	Safe target for bounded simulation.
Capture tool	tshark/Wireshark	Capture and analyze TCP flags.
Packet library	python3-scapy	Generate a small number of SYN packets.
Evidence	Provided PCAP or student capture	Forensic analysis source.
Optional second VM	Authorized host-only target	Demonstrate network capture without touching external networks.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab3
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 tshark wireshark python3-scapy
sudo systemctl enable --now apache2
# Optional supplied capture
wget -O evidence/mySYNFloodCapture.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-
  lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'
sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
```

Screenshot required: The Apache service state, evidence file details and SHA-256 hash.

```

(kali㉿kali)-[~/lab_assignments]
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~/lab_assignments]
$ cd ~/SBT-DF203-Lab3

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ pwd
/home/kali/SBT-DF203-Lab3

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ sudo apt install -y apache2 tshark wireshark python3-scapy
[sudo] password for kali:
apache2 is already the newest version (2.4.68-1).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
The following packages were automatically installed and are no longer required:
  gdal-data          libgeotiff5          libmuparser2v5      onboard-common
  gdal-plugins       libgirepository-1.0-1 libnetcdf22         proj-bin
  gir1.2-ayatanaappindicator3-0.1 libgpgme45          libodbc2           proj-data
  gir1.2-girepository-2.0 libhdf4-0           libodbcrc2         python3-fs
  libaec0            libhdf5-310         libodbcinst2       python3-gpg
  libarmadillo14     libhdf5-hl-310      libportmidi0       python3-talloc
  libarpac2t64       libjs-jquery-ui     libproj25          python3-tdb
  libavc1394-0       libjs-underscore   libpython3.13-dev  python3.13
  libblsc2-4         libjxr-tools        libraw23t64        python3.13-dev
  libcfitsio10t64    libjxr0t64          librttopo1         python3.13-minimal
  libcrypt-dev       libkmlbase1t64      libspatialite8t64  python3.13-venv
  libfreexl1         libkmlcore1t64      libwireshark18     rpcsvc-proto

Summary:
  Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 2130
  Download size: 2,002 kB
  Space needed: 764 kB / 54.2 GB available

Ign:1 http://http.kali.org/kali kali-rolling/main amd64 python3-scapy all 2.7.0+dfsg1-1
Get:1 http://http.kali.org/kali kali-rolling/main amd64 python3-scapy all 2.7.0+dfsg1-1 [2,002 kB]
Fetched 2,002 kB in 1min 34s (21.2 kB/s)
(Reading database ... 420331 files and directories currently installed.)
Preparing to unpack .../python3-scapy_2.7.0+dfsg1-1_all.deb ...
Unpacking python3-scapy (2.7.0+dfsg1-1) over (2.6.1+dfsg-1) ...
Setting up python3-scapy (2.7.0+dfsg1-1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.3.2) ...
Scanning processes ...
Scanning linux images ...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ █

```

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ wget -O evidence/mySYNFloodCapture.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'

--2026-09-11 22:05:32-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/
lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 2606:50c0:8002::154
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapture.pca 100%[=====>] 254.61K 343KB/s in 0.7s

2026-09-11 22:05:34 (343 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap

(kali㉿kali)-[~/SBT-DF203-Lab3]
```

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab3-Samuel_Chinedu_Onu
Trainee name	Samuel Chinedu Onu
Date and time started	11 th Sept 2026 / 3pm
Evidence file name(s)	evidence/mySYNFloodCapture.pcap
Source or generation method	
Original SHA-256	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 m
Working-copy SHA-256	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 m
Analysis workstation/VM	Kali Linux VM
Notes on any changes	Original captures preserved under `evidence/`

8. Part A - Establish a Normal Handshake Baseline

```
sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
sleep 2
curl --no-keepalive http://127.0.0.1/ >/dev/null
wait

tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/normal_handshake_flags.tsv
```

Document the expected SYN, SYN-ACK and ACK sequence before examining suspicious traffic.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w evidence/normal_http.pcapng &
[1] 20517

(kali@kali)-[~/SBT-DF203-Lab3]
$ [sudo] password for kali:

[1] + suspended (tty input) sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w
(kali@kali)-[~/SBT-DF203-Lab3]
$ sleep 2

(kali@kali)-[~/SBT-DF203-Lab3]
$ curl --no-keepalive http://127.0.0.1/ >/dev/null
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 10703 100 10703    0     0  766.4k      0    0     0      0     0      0
(kali@kali)-[~/SBT-DF203-Lab3]
$ wait

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn=1 || tcp.flags.fin=1' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
| tee reports/normal_handshake_flags.tsv

Warning: program compiled against libxml 215 using older 214
tshark: The file "evidence/normal_http.pcapng" doesn't exist.

```

Client → Server: SYN

Server → Client: SYN-ACK

Client → Server: ACK

(then HTTP data and FINs)

9. Part B - Conduct the Bounded Loopback Simulation

Create the following script exactly. It sends four SYN packets to the local Apache service and then exits. It does not run continuously and does not target another organization.

```

cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng

# Terminal 2, while capture is running
sudo python3 scripts/syn_probe_lab.py

```

Stop Condition

If the capture reaches 20 packets it stops automatically. Otherwise stop with Ctrl+C immediately after the four SYN packets and associated responses are captured.

```

Session  Actions  Edit  View  Help
(kali@kali)-[~]
$ cd ~/SBT-DF203-Lab3

(kali@kali)-[~/SBT-DF203-Lab3]
$ cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Terminal 1
sudo tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'Loopback: lo'
tshark: The file to which the capture would be saved ("evidence/bounded_syn_activity.pcapng") could not be opened: Permission denied.

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Terminal 2, while capture is running
sudo python3 scripts/syn_probe_lab.py
Sent 4 authorized training SYN packets to 127.0.0.1:80

(kali@kali)-[~/SBT-DF203-Lab3]
$

```

10. Part C - Identify SYN Indicators with tshark

```

PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt

# Initial SYNs only
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv

# SYN-ACK responses
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
  | tee reports/syn_ack_responses.tsv

# Resets and final ACK candidates
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv

```



```

(kali@kali)-[~/SBT-DF203-Lab3]
$ PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"

(kali@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
6412fd1aada5b0f0d1431a531721379d80c7e223096cefd2c515e2df690dd62 evidence/bounded_syn_activity.pcapng
6412fd1aada5b0f0d1431a531721379d80c7e223096cefd2c515e2df690dd62 working/bounded_syn_activity_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Initial SYNs only
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$ # SYN-ACK responses
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
  | tee reports/syn_ack_responses.tsv
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Resets and final ACK candidates
tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$

```

11. Part D - Quantify and Correlate the Activity

Counts by source and destination

```

tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt

```

Unique client source ports

```

tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt

```

Expert information and TCP analysis flags

```

tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

```

```

tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
  -T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv

```

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Counts by source and destination
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Unique client source ports
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$ # Expert information and TCP analysis flags
tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
  -T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv
Warning: program compiled against libxml 215 using older 214

(kali@kali)-[~/SBT-DF203-Lab3]
$

```


12. Part E - Compare Normal and Suspicious Sessions

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	1	4	Normal traffic starts with a single SYN.
SYN-ACK count	1	4	Server replies to every SYN
Completed handshakes	1	0	Full three-way handshake (SYN → SYN-ACK → ACK) only occurs in legitimate sessions.
Unique client source ports	1	4	Each SYN uses a different source port (RandShort).
HTTP request present?	Yes	No	Application-layer data only appears after a completed handshake.
RST packets	0 (or 1 at close)	0–4 (possible)	Kernel/Apache may eventually reset half-open sockets.
Observed duration	1-2 secs	A few seconds	Real SYN floods last much longer and occur at high rate.

A real SYN flood is defined by scale, rate, persistence and service impact. Four training packets are not a denial-of-service event; they reproduce the packet pattern in a safe way so that analysts can learn the indicators.